www.3aayaam.in

Spark Stream Processing Advanced

Agenda

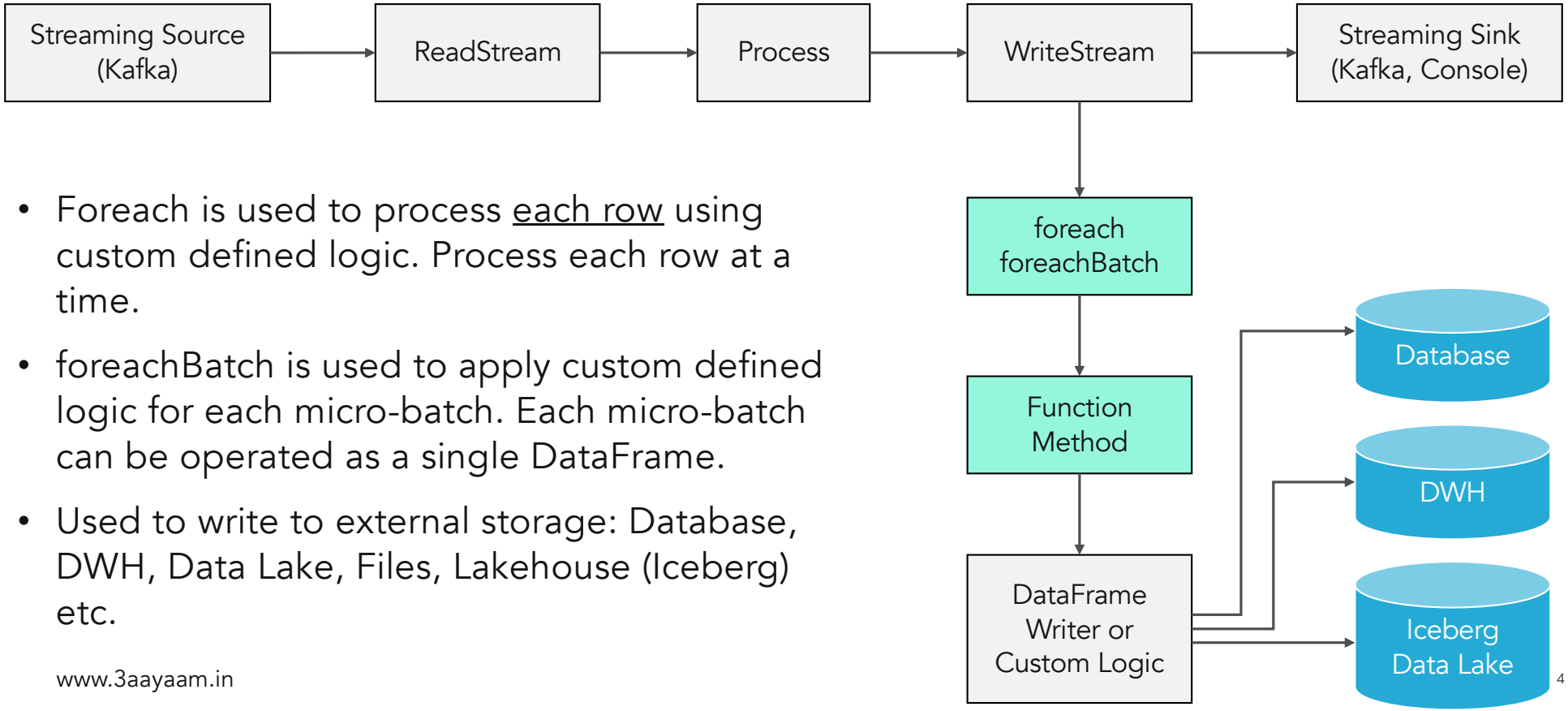
1. foreach, foreachBatch sink
2. Database/DWH Sink
3. Kafka Sink
4. Joining multiple Streams
5. Join Stream and Batch data
6. Available Now micro-batch
7. Continuous Processing

foreach, foreachBatch

- Console sink, Kafka sink.
- Database, DWH – Most important data stores/sinks.
- WriteStream can only be used with pre-defined sinks.

- How to write to databases, data lakehouses, data warehouses? There is no pre-defined sinks for these.
- How to write custom logic?

foreach, foreachBatch



- Foreach is used to process <u>each row</u> using custom defined logic. Process each row at a time.
- foreachBatch is used to apply custom defined logic for each micro-batch. Each micro-batch can be operated as a single DataFrame.
- Used to write to external storage: Database, DWH, Data Lake, Files, Lakehouse (Iceberg) etc.

Implementation - foreachBatch

- Best practice to use foreachBatch to write to Database, DWH etc.
- DataFrame writer can be used in the custom function to send to DB, DWH sinks.
- Write to multiple locations – DB + DWH + Data Lake + Iceberg.
- The python function will get DataFrame and micro-batch id as input.
- Micro-batch id can be used for exactly-once semantics.

“window” output

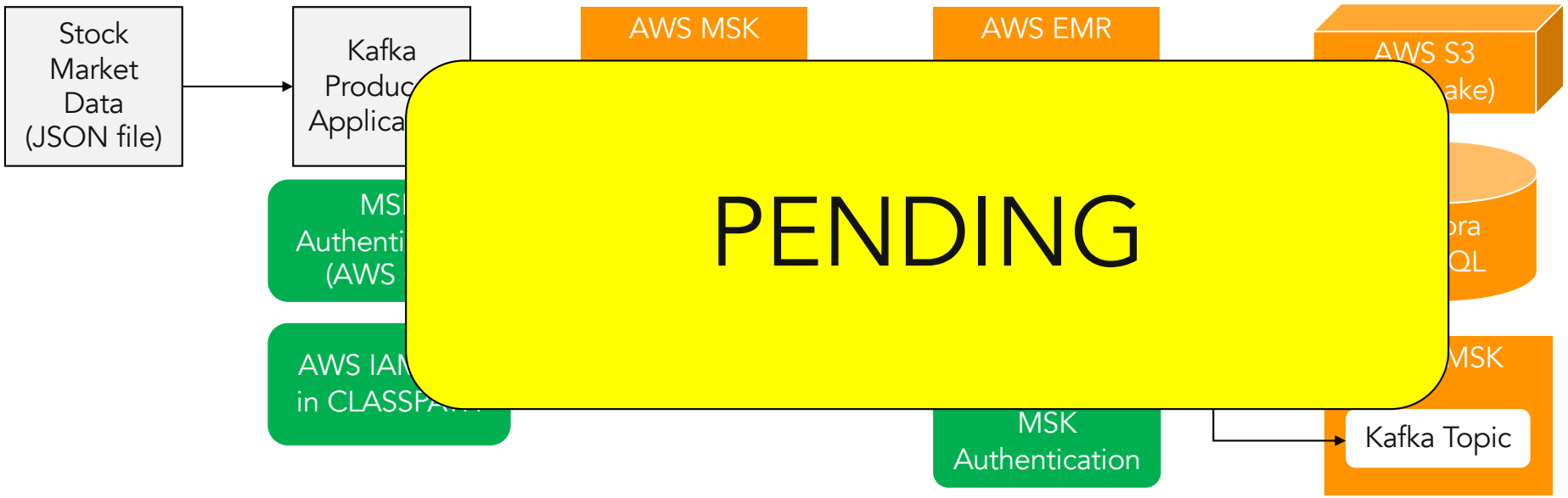
```
+----------------------------------------+
|window                                  |
+----------------------------------------+
|{2025-09-11 16:50:40, 2025-09-11 16:50:55}|
|{2025-09-11 16:49:10, 2025-09-11 16:49:25}|
|{2025-09-11 16:49:00, 2025-09-11 16:49:15}|
|{2025-09-11 16:51:10, 2025-09-11 16:51:25}|
|{2025-09-11 16:52:20, 2025-09-11 16:52:35}|
+----------------------------------------+
```

- window.start = 2025-09-11 16:50:40
- window.end = 2025-09-11 16:50:55

foreachBatch HandsOn

- Hands On.
- Find the no. of stocks bought & sold per country, company & stock exchange for the last 15 mins throughout the day and store that for near real-time operational reporting.
- Find the trading volume and price change for individual stocks across countries for real time reporting and future ad-hoc analytics.
- Find trade exposure by continuously calculating total value of BUY and SELL for Automobile industry in Germany.
- Find if there is any sudden surge in BUY or SELL for the following stocks – Mercedez Benz, BMW, Asia Commercial Bank and VinFast EV Motors.

foreachBatch HandsOn AWS



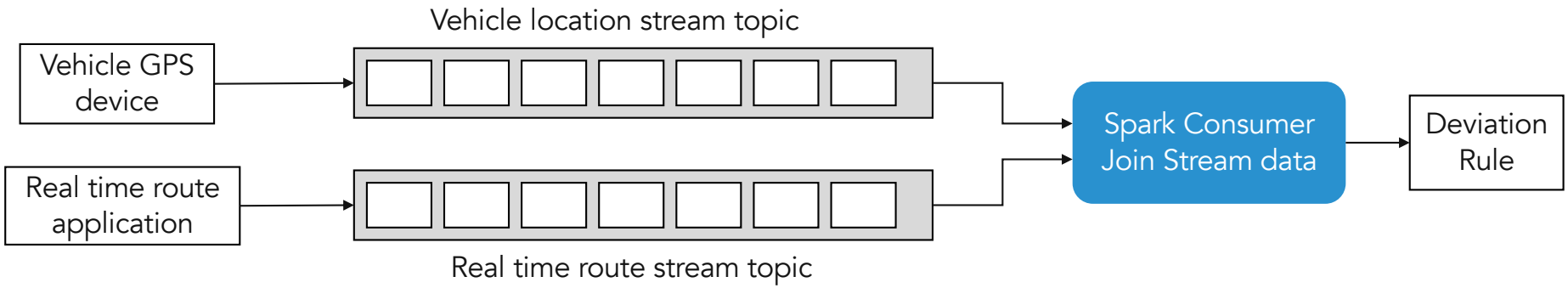
Implementation – foreach

- Foreach sink is used for single row (row-by-row) processing.
- Two implementation ways:
  - Python function
  - ForeachWriter Class
- DataFrame is not available

```
…writeStream.foreach(python_func)

def python_func(row):
    <processing-logic using Python>
```

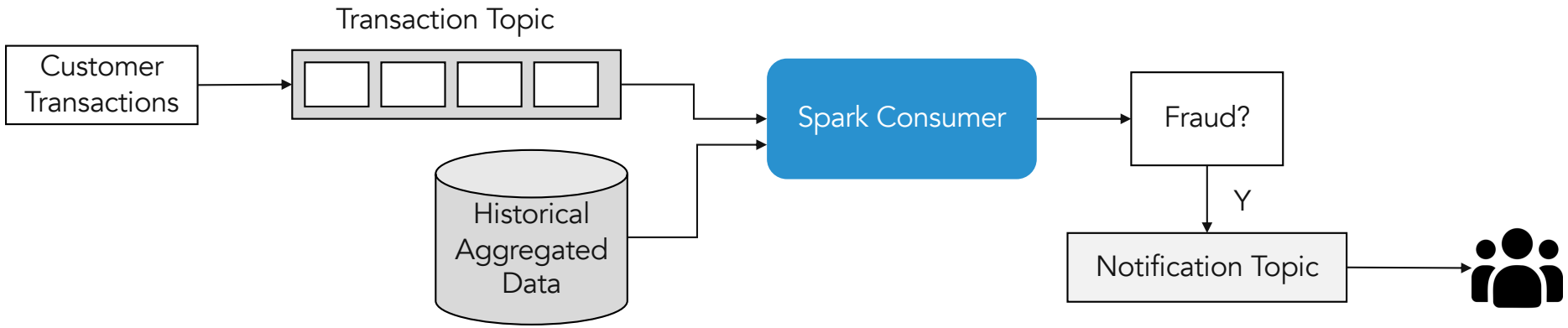
```
…writeStream.foreach(ForeachWriter( ))

class ForeachWriter:
    def open(self, partition, epoch):
        <open connection to db | open files etc.>

    def process(self, row):
        <processing logic using Python>

    def close(self, error):
        <close connection, files etc.>
```

Stream-to-Stream Joins



- Multiple Streams can be joined.
- Watermark & Event Time based predicate in WHERE clause should be provided.
- Only “append” & “complete” output modes are supported.
- Inner, Left, Right, Full joins.

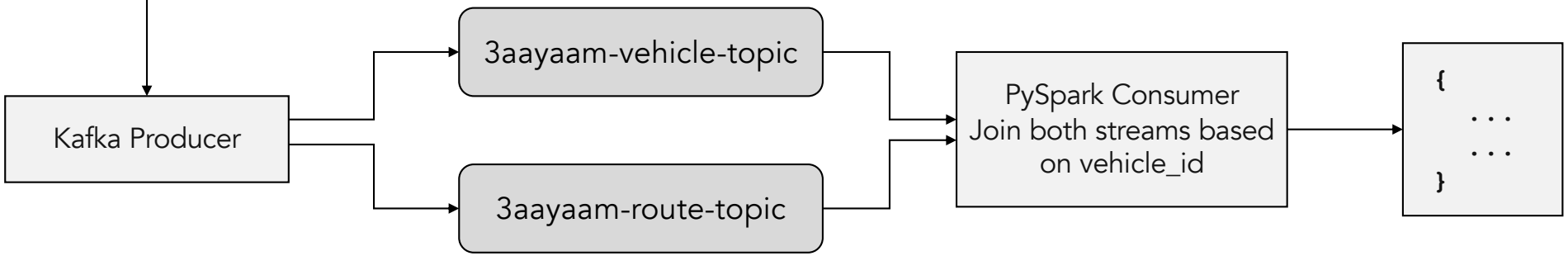
Stream-to-Batch Joins



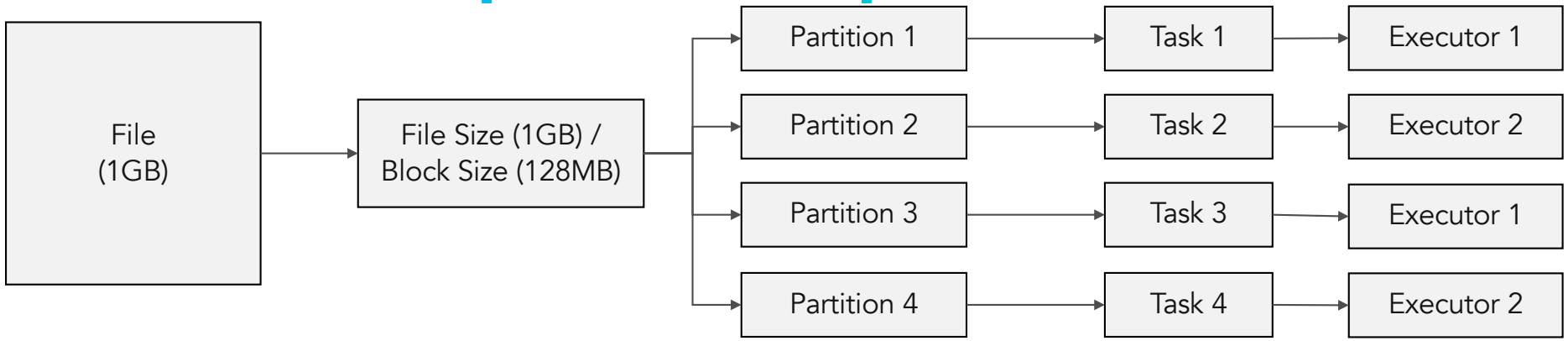
- Streams can be joined with batch data.
- No Watermarks are needed.
- Event Time based predicate is optional.
- All joins are supported.
- Hands On – Stream-to-stream join

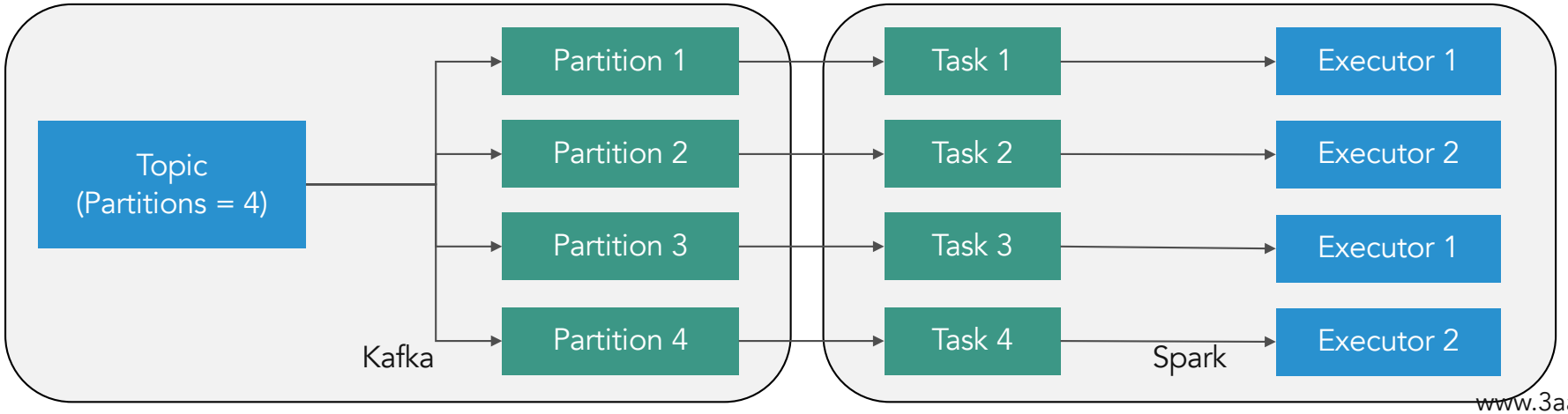
Multi-stream Join HandsOn


```
{
    "vehicle_id": "1",
    "vehicle_lat_location": -51.390732,
    "vehicle_long_location": -151.922251,
    "vehicle_timestamp": "2025-12-28 19:53:11.985645",
    "route_lat_location": -42.390732,
    "route_long_location": -137.922251,
    "route_timestamp": "2025-12-28 20:09:11.985645"
}
```

Kafka Topic & Spark Tasks





Available Now Trigger

- One time trigger.
- Process all available new data and then stop.
- writeStream...trigger(availableNow=True).start()

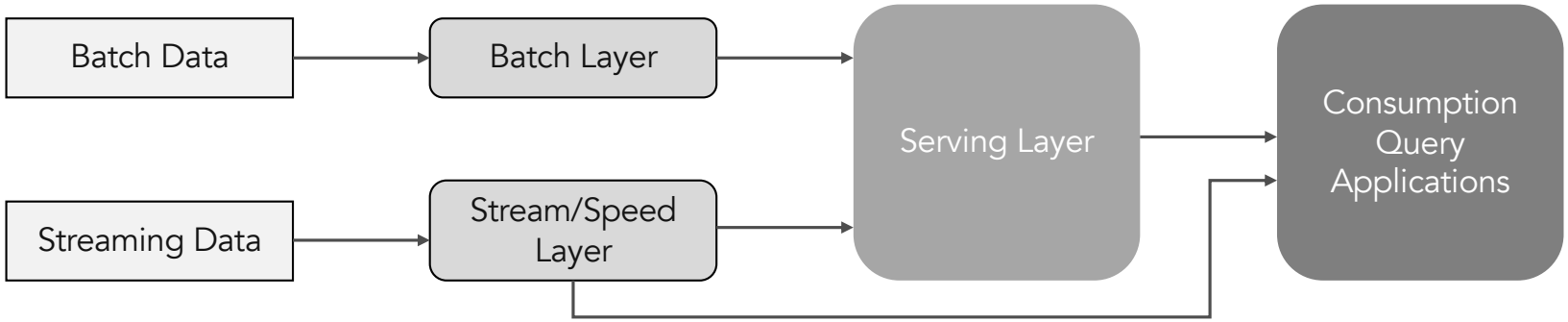
Continuous Processing

- Record-by-record processing. Process data as it arrives.
- For near real-time low latency processing.
  - Processing latency in micro-batch – 100 milliseconds or more.
  - Processing latency in continuous processing – Single or double digit millisecond.
- Implementation: writeStream...trigger(continuous="5 second").start()
- Considerations:
  - Only APPEND output mode.
  - checkpointLocation is mandatory.
  - Event time processing is not supported.
  - Aggregations, joins are not supported.
  - Kafka, Console, Memory sinks are supported.

Still Experimental

Lambda Architecture

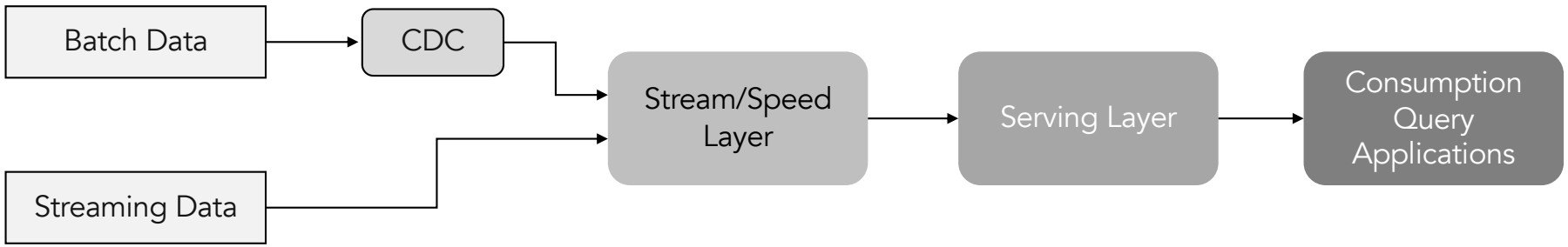
- Data processing architecture.
- Combines batch and stream processing.



- Separate batch & stream processing. Same logic has to be written multiple times.
- Complex architecture.

Kappa Architecture

- Everything is stream data.
- Same code is used for both batch and stream data.



- Single pipeline for both batch (historical data) & stream (real-time data).
- Simpler architecture, single stream processing layer.

Summary

www.3aayaam.in

1. foreach, foreachBatch sink
2. Stream-to-stream joins
3. Stream-to-batch joins
4. Available Now
5. Continuous Processing
6. Kappa & Lambda Architecture

Thank you for tuning in!!

This beautiful earth belongs to all. Let's live and let live.

www.3aayaam.in